

```
---
title: "Data Exploration"
output: html_document
---
```

Introduction: For this assignment, I analyzed a corpus of Early Modern political economy texts provided on Canvas. These documents were intentionally selected as part of a shared intellectual tradition, allowing for systematic comparison. After cleaning and normalizing the texts, I conducted an exploratory analysis using three approaches: TF-IDF to identify lexically distinctive terms, Pearson correlation to measure similarity between documents, and syntactic complexity measures to compare structural writing patterns across two selected texts. Together, these methods allowed me to examine how the texts relate to one another both in vocabulary use and grammatical structure.

```
```{r}
library(tidyverse)
library(tidytext)
library(stringr)
library(widyr)

```

```{r}
getwd()
```

```{r}
list.files("texts", full.names = TRUE)
```

```{r}
read_file("texts/A06785.txt")
```

```{r}

files <- list.files("texts",
 pattern = "\\..txt$",
 full.names = TRUE)

corpus_raw <- tibble(
 doc_id = tools::file_path_sans_ext(basename(files)),
 text = map_chr(files, read_file)
)

print(corpus_raw)

```{r}
# Count words that contain "f" before replacement
sum(str_detect(corpus_raw$text, "f"))
```
```

## Step 0: Normalization

In preparing the corpus for analysis, I adopted a minimal-intervention normalization strategy. The goal was to ensure lexical comparability across documents while preserving historically meaningful orthographic variation.

0.1) First, I replaced the Early Modern long S character (f) with the modern letter “s,” as required. Without this correction, words like “ftate” and “state” would be counted as different words, which would distort the analysis.

0.2) Second, I converted all text to lowercase and removed extra spaces. Lowercasing ensures that words such as “State” and “state” are treated as the same word. Collapsing extra whitespace prevents formatting inconsistencies from interfering with tokenization.

I chose not to modernize spelling or standardize variations such as u/v or archaic spellings. While doing so might make the texts look more uniform, it could also erase meaningful stylistic or historical differences between authors. Since this assignment focuses on exploring patterns rather than rewriting the texts, I aimed to preserve the original language as much as possible.

```
```{r}
corpus_normalized <- corpus_raw %>%
  mutate(
    text = str_replace_all(text, "f", "s") %>% # normalize long S
           str_replace_all("\\s+", " ") %>% # collapse extra whitespace
           str_to_lower() # standardize case
  )

corpus_normalized
```
```

During tokenization, I also removed punctuation, standalone numbers, common stopwords (like “the” and “and”), and tokens containing obvious OCR errors (such as ● symbols). These elements do not carry meaningful content for this type of analysis and would otherwise introduce noise.

Overall, I used a minimal cleaning approach: correcting only what was necessary for accurate comparison while preserving the integrity of the original texts.

```
```{r}
tokens_df <- corpus_normalized %>%
  unnest_tokens(word, text)
```

```{r}

tokens_df <- tokens_df %>%
  anti_join(stop_words, by = "word") %>% # remove stopwords
  filter(!str_detect(word, "^[0-9]+$")) %>% # remove numbers
  filter(!str_detect(word, "●")) # remove OCR artifacts

# tokens_df
```

```{r}
#Check document length balance
tokens_df %>%
  count(doc_id, sort = TRUE)
```

```{r}
# Check most frequent words

tokens_df %>%
  count(word, sort = TRUE) %>%
  slice_max(n, n = 20)
```

```{r}
# Vocabulary size per document
```

```
tokens_df %>%
  group_by(doc_id) %>%
  summarise(unique_words = n_distinct(word))
```



```
```{r}
Check for any non-alphabetic tokens

tokens_df %>%
 filter(str_detect(word, "[^a-z]")) %>%
 count(word, sort = TRUE)
```
```


```

These are real words and carry meaning, not OCR errors.

#### APPROACH 1: TF – IDF

I computed TF-IDF scores for each document to identify terms that are distinctive relative to the rest of the corpus. This allowed me to see what makes each text lexically unique while still operating within a shared vocabulary.

```
```{r}
# Step 1: Document feature matrix

dfm_counts <- tokens_df %>%
  count(doc_id, word, sort = TRUE)

dfm_counts
```



```
```{r}
Step 2: Compute TF-IDF weights

tf_idf_data <- dfm_counts %>%
 bind_tf_idf(word, doc_id, n) %>%
 arrange(desc(tf_idf))

tf_idf_data
```



```
```{r}
# Step 3: Top distinctive terms per document

top_terms <- tf_idf_data %>%
  group_by(doc_id) %>%
  slice_max(tf_idf, n = 15) %>%
  ungroup()

top_terms
```



```
```{r}
top_terms <- tf_idf_data %>%
 group_by(doc_id) %>%
 slice_max(tf_idf, n = 15) %>%
 ungroup()
```



```
```{r}
colnames(top_terms)
```
```


```


```


```


```

```

```{r}
# Export one TF-IDF table per document

# Split TF-IDF results by document
doc_list <- split(top_terms, top_terms$doc_id)

dir.create("tfidf_results", showWarnings = FALSE)

purrr::walk(names(doc_list), function(doc) {
  write_csv(doc_list[[doc]],
            file.path("tfidf_results",
                      paste0("tf_idf_", doc, ".csv")))
})
```

```{r}
# Create folder for TF-IDF distinctiveness results

dir.create("tfidf_distinctiveness", showWarnings = FALSE)

# Split results by document
doc_list <- split(top_terms, top_terms$doc_id)

# Export each document's distinctive terms
purrr::walk(names(doc_list), function(doc) {
  write_csv(doc_list[[doc]],
            file.path("tfidf_distinctiveness",
                      paste0("distinctive_terms_", doc, ".csv")))
})
```

```{r}
list.files("tfidf_distinctiveness")
```

```

## APPROACH 2 – Pearson Correlation

Using a document–feature matrix, I calculated pairwise Pearson correlations between documents to measure similarity in vocabulary distributions. I visualized these relationships with a heatmap and identified the most and least similar document pairs.

```

```{r}
# Load required libraries

library(quanteda)
library(quanteda.textstats)
library(tidyverse)

```

```{r}
# Create quanteda corpus

corpus_quanteda <- corpus(corpus_normalized, text_field = "text")
#corpus_quanteda
```

```{r}
# Tokenize

tokens_quanteda <- tokens(
  corpus_quanteda,
  remove_punct = TRUE,

```

```

    remove_numbers = TRUE
  )
  ...

  ```{r}
 # Remove stop words
 tokens_quanteda <- tokens_remove(tokens_quanteda, stopwords("en"))

 #tokens_quanteda
  ```

  ```{r}
 # Create DFM
 dfm_counts <- dfm(tokens_quanteda)

 dfm_counts
  ```

  ```{r}
 dfm_trimmed <- dfm_trim(dfm_counts, min_termfreq = 5)

 dfm_trimmed
  ```

  ```{r}
 sim_r <- textstat_simil(dfm_trimmed,
 margin = "documents",
 method = "correlation")

 r_mat <- as.matrix(sim_r)

 r_mat <- round(r_mat, 3)

 r_mat[1:5, 1:5]
  ```

  ```{r}
 heat_df <- as.data.frame(r_mat) %>%
 rownames_to_column("doc_i") %>%
 pivot_longer(~doc_i,
 names_to = "doc_j",
 values_to = "r") %>%
 filter(doc_i != doc_j)

 # Most similar pairs
 most_similar <- heat_df %>%
 arrange(desc(r)) %>%
 distinct(pmin(doc_i, doc_j),
 pmax(doc_i, doc_j),
 .keep_all = TRUE) %>%
 slice_head(n = 2)

 most_similar
 ...

  ```{r}
  # Least similar pairs
  least_similar <- heat_df %>%
    arrange(r) %>%
    distinct(pmin(doc_i, doc_j),
              pmax(doc_i, doc_j),
              .keep_all = TRUE) %>%
    slice_head(n = 2)

```

```

least_similar
```
```{r}
ggplot(heat_df,
      aes(x = doc_j,
          y = doc_i,
          fill = r)) +
  geom_tile() +
  coord_fixed() +
  scale_fill_gradient2(
    low = "#2C7BB6",      # deep blue
    mid = "#FFFFBF",      # yellow
    high = "#D7191C",     # dark red
    midpoint = 0.5
  ) +
  labs(title = "Pearson Correlation Between Documents",
       x = NULL,
       y = NULL,
       fill = "Correlation") +
  theme_minimal() +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1),
    panel.grid = element_blank()
  )
```

```

### APPROACH 3 – Syntactic complexity profile

Based on the similarity results, I selected two texts and implemented the Week 05 syntactic complexity framework. I measured sentence length, clause density, dependent clauses, coordination, and complex nominals to compare how the texts differ in grammatical structure.

```

```{r}
library(udpipe)
library(dplyr)
library(stringr)

```{r}
model <- udpipe_download_model(language = "english")
ud_model <- udpipe_load_model(model$file_model)

```{r}
#Extract two texts

text_subset <- corpus_normalized %>%
  filter(doc_id %in% c("A32827", "A93819"))

```{r}
parsed <- udpipe_annotate(ud_model,
 x = text_subset$text,
 doc_id = text_subset$doc_id)

parsed_df <- as.data.frame(parsed)

```{r}
# MLS

sentence_lengths <- parsed_df %>%
  group_by(doc_id, sentence_id) %>%

```

```

    summarise(words = n(), .groups = "drop") %>%
    group_by(doc_id) %>%
    summarise(MLS = mean(words))
  },
  `r`
# Clauses per sentence

clauses <- parsed_df %>%
  filter(upos == "VERB") %>%
  group_by(doc_id) %>%
  summarise(total_clauses = n())
  },
  `r`
sentences <- parsed_df %>%
  distinct(doc_id, sentence_id) %>%
  count(doc_id, name = "total_sentences")
  },
  `r`
cs_ratio <- clauses %>%
  left_join(sentences, by = "doc_id") %>%
  mutate(C_per_S = total_clauses / total_sentences)
  },
  `r`
colnames(parsed_df)
  },
  `r`
# Dependent Clauses

dependent_clauses <- parsed_df %>%
  filter(dep_rel %in% c("ccomp", "xcomp", "advcl", "acl")) %>%
  group_by(doc_id) %>%
  summarise(dep_clauses = n())
  },
  `r`
# Coordination

coordination <- parsed_df %>%
  filter(dep_rel == "conj") %>%
  group_by(doc_id) %>%
  summarise(coordination = n())
  },
  `r`
# Complex Nominals

complex_nominals <- parsed_df %>%
  filter(dep_rel %in% c("amod", "nmod", "compound")) %>%
  group_by(doc_id) %>%
  summarise(complex_nominals = n())
  },
  `r`
summary_table <- sentence_lengths %>%
  left_join(cs_ratio, by = "doc_id") %>%
  left_join(dependent_clauses, by = "doc_id") %>%
  left_join(coordination, by = "doc_id") %>%

```

```

left_join(complex_nominals, by = "doc_id")

summary_table
```

```{r}
# Extract example sentences
example_sentences <- parsed_df %>%
  group_by(doc_id, sentence_id) %>%
  summarise(sentence = paste(token, collapse = " "),
    .groups = "drop") %>%
  group_by(doc_id) %>%
  slice_sample(n = 1)
```

```{r}
dir.create("syntactic_results", showWarnings = FALSE)

write_csv(example_sentences,
  file.path("syntactic_results",
    "syntactic_example_sentences.csv"))
```

```{r}
list.files("syntactic_results")
```

```{r}
sentences_full <- parsed_df %>%
  group_by(doc_id, sentence_id) %>%
  summarise(
    sentence = paste(token, collapse = " "),
    word_count = n(),
    .groups = "drop"
  )

example_A32827 <- sentences_full %>%
  filter(doc_id == "A32827") %>%
  slice_max(word_count, n = 1)

example_A93819 <- sentences_full %>%
  filter(doc_id == "A93819") %>%
  slice_min(word_count, n = 1)

example_sentences <- bind_rows(example_A32827,
  example_A93819)

example_sentences
```

```{r}
syntactic_measures <- summary_table %>%
  mutate(
    DC_per_S = dep_clauses / total_sentences,
    DC_per_C = dep_clauses / total_clauses,
    Coord_per_S = coordination / total_sentences,
    Coord_per_C = coordination / total_clauses,
    CN_per_S = complex_nominals / total_sentences,
    CN_per_C = complex_nominals / total_clauses
  ) %>%
  select(doc_id, MLS, C_per_S,
    DC_per_S, DC_per_C,
    Coord_per_S, Coord_per_C,
    CN_per_S, CN_per_C)
```

```



```

```{r}
syntactic_long <- syntactic_measures %>%
  pivot_longer(-doc_id,
               names_to = "measure",
               values_to = "value")
...

```{r}
ggplot(syntactic_long,
 aes(x = measure,
 y = value,
 fill = doc_id)) +
 geom_col(position = "dodge") +
 facet_wrap(~ measure, scales = "free") +
 scale_fill_manual(values = c("#4B3F72", "#1B998B")) +
 labs(title = "Syntactic Complexity Profile",
 x = NULL,
 y = "Value",
 fill = "Document") +
 theme_minimal() +
 theme(axis.text.x = element_blank(),
 strip.text = element_text(face = "bold"))
...

```

### Central Analytical Question

Do lexical similarity patterns within the corpus correspond to shared structural (syntactic) complexity, or are vocabulary and sentence structure operating independently?

This convergence across methods suggests that variation within the corpus reflects meaningful differences in intellectual style rather than random fluctuation. Lexical clustering (Pearson), distinctiveness (TF-IDF), and syntactic density (complexity measures) all point toward internal differentiation in argumentative strategy and rhetorical construction.

Importantly, Pearson correlation measures similarity in patterns of language use – not causation – but when combined with syntactic evidence, it strengthens the interpretation that the corpus contains distinct stylistic groupings within a shared intellectual tradition.